



CRACKUCCINO

SUBSTITUTION CIPHER DECRYPTOR

MPI

C

Brute Force

crackuccino write-up

November 11th, 2025

© 2025 Paul Mancion. Please do not redistribute without permission.

<https://github.com/MindMidas/crackuccino>

Program Descriptions

src/encrypt.c:

Reads plaintext, builds a unique input_dict of letters, and shuffles it to form an encryption key. Each letter in the plaintext is replaced according to this mapping (non-letters stay the same). Outputs ciphertext and the key.

src/decrypt_serial.c:

Reads ciphertext and dictionary, builds input_dict of unique letters, and loads HashDict (for word lookup). Uses a recursive permutation search over all possible letter mappings, decrypting each permutation and checking if all words exist in HashDict. Valid plaintexts are stored in HitBuffer (custom HitList data struct) and printed with runtime stats at the end.

src/decrypt_mpi.c:

Rank 0 reads ciphertext, broadcasts it, and all ranks build the same HashDict (could be improved by broadcasting). Work is split by prefix-depth (uses $-d$ flag if given + valid, else auto-chooses d) permutation chunks assigned via $task_{id} \% size$. Each rank decrypts and validates locally, storing hits in HitBuffer (custom HitList data struct). The results (hits and stats) are gathered and printed by rank 0 at the end.

Testing and Analysis Process

A structured set of experiments and a library was written (see *tests/python/...*) to confirm the correctness, stability, and performance scaling of the programs. Initial tests used Valgrind to ensure memory safety, followed by input robustness checks to verify consistent handling of mixed-case text, punctuation, and symbols. Further tests validated dictionary matching accuracy and confirmed that both serial and MPI versions explored all expected permutations. The performance analysis included a baseline, strong-scaling, depth-variation, and break-even tests to measure runtime, speedup, and efficiency across different ranks and problem sizes. It is worth noting that these tests were written for and run on a machine with 8 cores and max $np = 8$. They would require modification to see similar results on another machine.

Correctness of Programs

Test 1-3 (Memory Safety and Stability):

Paths:

*tests/python/t1.py,
tests/python/t2.py,
tests/python/t3.py*

Description:

Test 1 to 3 run the command-line programs (encrypt, decrypt-serial, decrypt-mpi) with Valgrind's memory check tool to confirm that each program manages memory correctly. The goal is to ensure there are no leaks or invalid memory accesses, confirming correctness and stability before performance testing and analysis.

Test Results and Observations:

```
{
  "name": "T1",
  "category": "Correctness of Programs",
  "section": "Memory Safety & Stability",
  "case": "Encrypt",
  "passed": true,
  "plaintext": "You merely adopted the dark",
  "input": {
    "input_dict": "youmerladpthk",
    "encrypt_dict": "ulkhrdypoamte",
    "ciphertext": "ulk hrdryu polamro mtr opde"
  },
  "valgrind": {
    "error_summary": null,
    "definitely_lost_bytes": null,
    "indirectly_lost_bytes": null,
    "possibly_lost_bytes": null,
    "still_reachable_bytes": null,
    "raw": "..."
  }
}
```

```
{
  "name": "T2",
  "category": "Correctness of Programs",
  "section": "Memory Safety & Stability",
  "case": "Serial Decrypt",
  "passed": true,
  "plaintext": "Swear to me",
  "valgrind_enabled": true,
  "reps": 3,
  "detected_cores": 8,
  "total_ranks": 1,
  "ciphertext": "raost ew mo",
  "n_letters": 8,
  "depth": 1,
  "expected_perms": 40320,
  "visited_perms": 40320,
  "valid_hits": 176,
  "found": [...],
  "configs": [
    {
      "ranks": 1,
      "depth": 1,
      "times_sec": [...],
      "summary": {...},
      "valgrind": {
        "error_summary": null,
        "definitely_lost_bytes": null,
        "indirectly_lost_bytes": null,
        "possibly_lost_bytes": null,
        "still_reachable_bytes": null,
        "raw": "..."
      }
    }
  ]
}
```

```

{
  "name": "T3",
  "category": "Correctness of Programs",
  "section": "Memory Safety & Stability",
  "case": "MPI Decrypt",
  "passed": true,
  "plaintext": "Why do we fall",
  "valgrind_enabled": true,
  "reps": 3,
  "detected_cores": 8,
  "total_ranks": 4,
  "ciphertext": "afy dl ae wohh",
  "n_letters": 9,
  "depth": 1,
  "expected_perms": 362880,
  "visited_perms": 362880,
  "valid_hits": 145,
  "found": [...],
  "configs": [
    {
      "ranks": 1,
      "depth": 1,
      "times_sec": [...],
      "summary": {...},
      "valgrind": {
        "error_summary": null,
        "definitely_lost_bytes": null,
        "indirectly_lost_bytes": null,
        "possibly_lost_bytes": null,
        "still_reachable_bytes": null,
        "raw": "..."
      }
    },
    ...
    {
      "ranks": 8,
      "depth": 2,
      "times_sec": [...],
      "summary": {...},
      "valgrind": {
        "error_summary": null,
        "definitely_lost_bytes": null,
        "indirectly_lost_bytes": null,
        "possibly_lost_bytes": null,
        "still_reachable_bytes": null,
        "raw": "..."
      }
    }
  ]
}

```

As shown in the test results above, all programs passed the Valgrind memory tests with no errors or memory leaks detected. Outside of MPI noise (leaks, etc.), the summaries showed that all allocated code (that does not include MPI) memory was freed properly, and no invalid memory accesses occurred during the execution of the programs. This confirms that the encrypt, serial decrypt, MPI decrypt, hash dictionary, and valid hits dynamic data struct were implemented safely. Thus, memory management across the programs seems stable and meets the project requirements.

Test 4-6 (Input Robustness & Normalization):

Paths:

*tests/python/t4.py,
tests/python/t5.py,
tests/python/t6.py*

Note: Replicating tests for decrypt-serial are unnecessary as decrypt-mpi shares the same library implementation.

Description:

Tests 4 to 6 evaluate decrypt-mpi for result consistency, correctness, and robustness when handling different plaintext inputs. Each test case introduces a specific variation in input formatting to confirm that the program's logic and decryption accuracy remain unaffected:

- T4 (Case Insensitivity): Ensure that decrypt-mpi correctly processes plaintext containing mixed uppercase and lowercase letters by converting output to lowercase.
- T5 (Punctuation): Ensure that non-alphabetic characters (apostrophe, full stop) are preserved and do not interfere with permutation logic or result summaries.
- T6 (Non-alphabetic characters): Ensure correctness when the plaintext includes digits or symbols. They must be preserved in the output string if they are not part of the word. Non-alphabetic characters (excluding apostrophe) signify the end of a word in my program. This project uses that rule to define a clear policy for non-alphabetic characters outside of the dictionary.

The goal is to confirm that such characters are ignored in permutation generation but preserved in the decrypted output. Hence, implementing input normalization and maintaining overall text integrity.

Test Results and Observations:

```
{
  "name": "T4",
  "category": "Correctness of Programs",
  "section": "Input Robustness & Normalization",
  "case": "Case Insensitivity",
  "passed": true,
  "plaintext": "Batman Has No Limits",
  "valgrind_enabled": false,
  "reps": 2,
  "detected_cores": 8,
  "total_ranks": 1,
  "ciphertext": "mnbino anl os htitbl",
  "n_letters": 10,
  "depth": 2,
  "expected_perms": 3628800,
  "visited_perms": 3628800,
  "valid_hits": 2,
  "found": [
    "[rank 2] [permutation: batmnoshli] found: batman oas nh limits",
    "[rank 2] [permutation: batmnhsoili] found: batman has no limits"
  ],
  "configs": [
    {
      "ranks": 8,
      "depth": 2,
      "times_sec": [...],
      "summary": {...}
    }
  ]
}
```

```
{
  "name": "T5",
  "category": "Correctness of Programs",
  "section": "Input Robustness & Normalization",
  "case": "Punctuation",
  "passed": true,
  "plaintext": "Vengeance! isn't, power.",
  "valgrind_enabled": false,
  "reps": 2,
  "detected_cores": 8,
  "total_ranks": 1,
  "ciphertext": "spnapvnrp! wen'o, igtpc.",
  "n_letters": 13,
  "depth": 2,
  "expected_perms": 6227020800,
  "visited_perms": 6227020800,
  "valid_hits": 5,
  "found": [
    "[rank 4] [permutation: vengacwotpris] found: vengeance! won't, pries.",
    "[rank 4] [permutation: vengacpistown] found: vengeance! pin's, tower.",
    "[rank 4] [permutation: vengacispowr] found: vengeance! isn't, power.",
    "[rank 4] [permutation: vengactoswipr] found: vengeance! ton's, wiper.",
    "[rank 4] [permutation: vengactispowr] found: vengeance! tin's, power."
  ],
  "configs": [
    {
      "ranks": 8,
      "depth": 2,
      "times_sec": [...],
      "summary": {...}
    }
  ]
}
```

```
{
  "name": "T6",
  "category": "Correctness of Programs",
  "section": "Input Robustness & Normalization",
  "case": "Non-alphabetic characters",
  "passed": true,
  "plaintext": "$123! !I'm! :not# (afraid). $123!",
  "valgrind_enabled": false,
  "reps": 2,
  "detected_cores": 8,
  "total_ranks": 1,
  "ciphertext": "$123! !m'd! :fit# (oraomn). $123!",
  "n_letters": 9,
  "depth": 2,
  "expected_perms": 362880,
  "visited_perms": 362880,
  "valid_hits": 2,
  "found": [
    "[rank 3] [permutation: imtonafrd] found: $123! !i'm! :ton# (afraid). $123!",
    "[rank 3] [permutation: imnotafrd] found: $123! !i'm! :not# (afraid). $123!"
  ],
  "configs": [
    {
      "ranks": 8,
      "depth": 2,
      "times_sec": [...],
      "summary": {...}
    }
  ]
}
```

As shown in the test results, Tests 4 to 6 all passed successfully, which confirms that decrypt-mpi maintains consistent and correct output across varying plaintext formats. In T4 (Case Insensitivity), the program correctly normalized the mixed-case plaintext, decrypting and preserving lowercase consistency. In T5 (Punctuation), non-alphabetic characters such as apostrophes and full stops were preserved in the decrypted text. Finally, T6 (Non-Alphabetic Characters) verified that digits and symbols within the plaintext were preserved in their correct positions without disrupting the decryption. Thus, the programs ensure input normalization and preserve overall text integrity.

Test 7-8 (Result Verification & Dictionary Matching):

Paths:

`tests/python/t7.py,`
`tests/python/t8.py,`

Description:

These tests use the project's small dictionary (`utils.CUSTOM_DICT_PATH`) to verify result-counting and input normalization.

- T7 (Valid match detection): plaintext contains a phrase with all words present in the small dictionary. We assert that the program reports *valid_hits* = 1 and lists the matching line in *found* list.
- T8 (No match detection): plaintext includes characters that prevent a dictionary match. We confirm with *valid_hits* = 0 and an empty *found* list.

The goal here is to ensure dictionary matching and *valid_hits* accounting is correct in both positive and negative control scenarios.

Test Results and Observations:

```
{
  "name": "T7",
  "category": "Correctness of Programs",
  "section": "Result Verification & Dictionary Matching",
  "case": "Valid match detection",
  "passed": true,
  "plaintext": "hello world",
  "valgrind_enabled": false,
  "reps": 2,
  "detected_cores": 8,
  "total_ranks": 1,
  "ciphertext": "oeddr wrhdl",
  "n_letters": 7,
  "depth": 2,
  "expected_perms": 5040,
  "visited_perms": 5040,
  "valid_hits": 1,
  "found": [
    "[rank 5] [permutation: helowrd] found: hello world"
  ],
  "configs": [
    {
      "ranks": 8,
      "depth": 2,
      "times_sec": [...],
      "summary": {...}
    }
  ]
}
```

```

{
  "name": "T8",
  "category": "Correctness of Programs",
  "section": "Result Verification & Dictionary Matching",
  "case": "No match detection",
  "passed": true,
  "plaintext": "hello wurd",
  "valgrind_enabled": false,
  "reps": 2,
  "detected_cores": 8,
  "total_ranks": 1,
  "ciphertext": "uohhw rldhe",
  "n_letters": 8,
  "depth": 2,
  "expected_perms": 40320,
  "visited_perms": 40320,
  "valid_hits": 0,
  "found": [],
  "configs": [
    {
      "ranks": 8,
      "depth": 2,
      "times_sec": [...],
      "summary": {...}
    }
  ]
}

```

As shown in the test results above, both T7 (positive control) and T8 (negative control) passed. In T7, *valid_hits* = 1 as expected, and the found array contains a single decrypted entry with the string “hello world”. On the other hand, in T8, *valid_hits* = 0, and found is empty. This is expected as the word “wurd” is invalid. Hence, the serial and MPI hash-dictionary matching logic is behaving correctly; it identifies exact (normalized) word matches when they exist and ensures that all words in the string must be valid for the plaintext to be valid.

Tests 1-8 (Permutation Consistency):

Description, Results, and Observations:

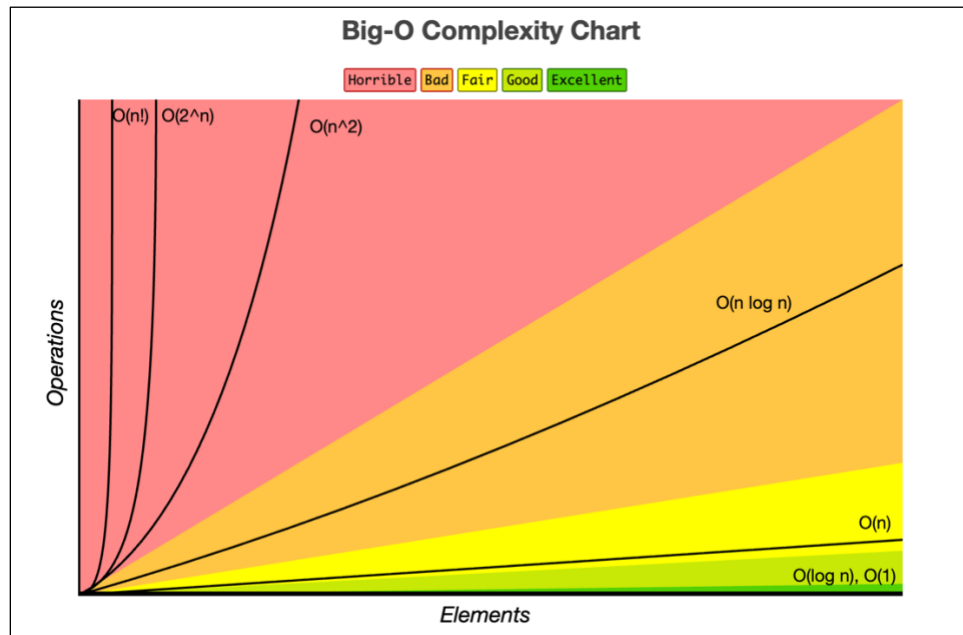


Figure 1: Big-O Complexity Chart (Big-O Cheat Sheet, n.d.)

Big-O Cheat Sheet. (n.d.). Big-O Complexity Chart. Retrieved November 11, 2025, from <https://www.bigocheatsheet.com/>

This test ensures that the expected number of permutations (*expected_perms*) generated from the input dictionary and plaintext matches the visited permutations (*visited_perms*) actually processed by both the serial (decrypt-serial) and MPI (decrypt-mpi) decrypt programs. For all completed tests (T1-T8), these two values were equal, indicating that:

- Every possible permutation ($n!$, where n is number of unique letters) derived from the unique letters of the encrypted text was correctly visited during the brute-force decryption.
- The permutation traversal logic behaves deterministically across different rank counts (1 to 8).
- Both the serial and MPI implementations share identical permutation logic for searching.

As shown in Figure 1, the factorial time complexity $O(n!)$ lies within the "Horrible" region of the Big-O Complexity Chart. This means the number of operations grows extremely fast with each new unique letter. This graph reinforces why the permutation tests become computationally expensive n increases and why MPI parallelization is beneficial for distributing the workload across ranks. Hence, the programs demonstrate correctness and reliability for this project.

Performance Scaling Analysis

Test 9 (Baseline):

Paths:

tests/python/t9.py,

Description:

Test 9 establishes the performance baseline of the serial (decrypt-serial) and MPI (decrypt-mpi) decryption programs when executed with a single process/rank ($np = 1$) on fixed input (same plaintext and dictionary). Although the ciphertext will differ due to the random shuffle, the brute-force approaches ensure both programs generate the same number of permutations (identical workloads). Each test runs 30 repetitions to capture variability in total runtime:

- T9a: executes the serial version to measure the runtime.
- T9b: executes the MPI version with $np = 1$, where message-passing layers are initialized but not distributed, allowing for some understanding of MPI overhead on a single process.

The goal is to verify that both decryption implementations behave equivalently under identical conditions, producing the same output and similar runtime distributions. This establishes a reference point for later scaling tests. It would help confirm that MPI introduces minimal overhead when run as a single process, and that any future speedup arises from parallelization rather than implementation differences.

Test Results and Observations:

```
{
  "name": "T9a",
  "category": "Performance Scaling Analysis",
  "section": "Baseline",
  "case": "Serial baseline (decrypt-serial, np=1)",
  "passed": true,
  "plaintext": "I'm not afraid. I'm angry.",
  "valgrind_enabled": false,
  "reps": 30,
  "detected_cores": 8,
  "total_ranks": 1,
  "ciphertext": "a'f tod rgyran. a'f rtiym.",
  "n_letters": 11,
  "depth": 1,
  "expected_perms": 39916800,
  "visited_perms": 39916800,
  "valid_hits": 1,
  "found": [
    "[rank 0] [permutation: imnotafrdgy] found: i'm not afraid. i'm angry."
  ],
  "configs": [
    {
      "ranks": 1,
      "depth": 1,
      "times_sec": [2.362367, ..., 2.345487],
      "summary": {...}
    }
  ]
}
```

```

{
  "name": "T9b",
  "category": "Performance Scaling Analysis",
  "section": "Baseline",
  "case": "MPI baseline (decrypt-mpi, np=1)",
  "passed": true,
  "plaintext": "I'm not afraid. I'm angry.",
  "valgrind_enabled": false,
  "reps": 30,
  "detected_cores": 8,
  "total_ranks": 1,
  "ciphertext": "a'f toy rgirad. a'f rtnim.",
  "n_letters": 11,
  "depth": 1,
  "expected_perms": 39916800,
  "visited_perms": 39916800,
  "valid_hits": 1,
  "found": [
    "[rank 0] [permutation: imnotafndgy] found: i'm not afraid. i'm angry."
  ],
  "configs": [
    {
      "ranks": 1,
      "depth": 1,
      "times_sec": [2.41681, ..., 2.380251],
      "summary": {...}
    }
  ]
}

```

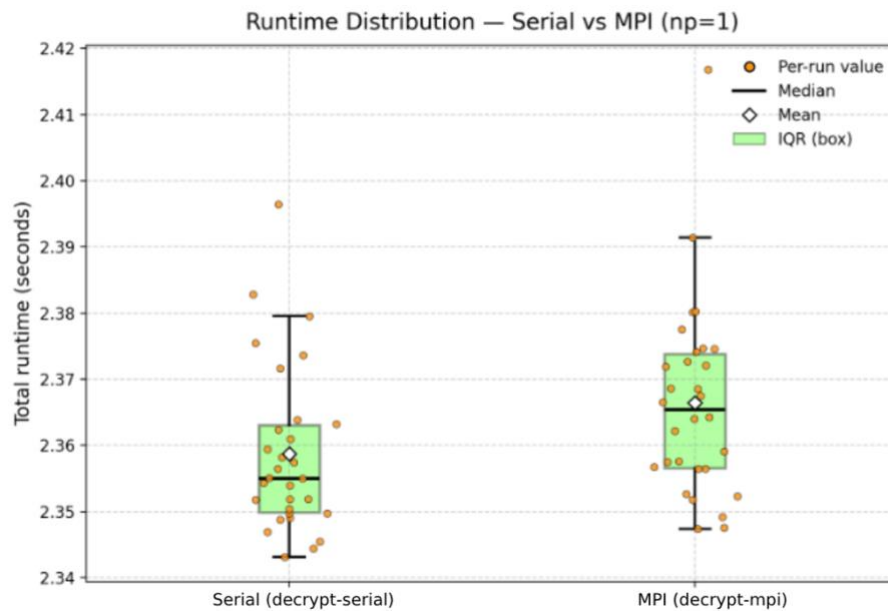


Figure 2: Runtime Distribution Comparison Between Serial and MPI Decrypt Implementations ($np = 1$)

The results show that both the serial (decrypt-serial) and MPI (decrypt-mpi) implementations produced nearly identical runtime distributions when executed with a single process ($np = 1$). The mean and median values overlap closely. This indicates minimal performance difference and confirms that MPI introduces little overhead in single-rank execution. This validates that both implementations perform almost the same under identical workloads and establishes a reliable baseline for the next scaling tests, where performance differences can be linked to parallelization rather than implementation inefficiencies.

Test 10 (Strong Scaling):

Paths:

`tests/python/t10.py`

Description:

Test 10 evaluates the strong scaling performance of decrypt-mpi by keeping the search space constant (12!) while increasing the number of MPI ranks. The ciphertext is fixed with 12 unique letters. The test is executed for each $np = [1, 2, 3, 4, 5, 6, 7, 8]$. Each np configuration has 30 repetition runs to capture stable runtime averages and variance. By keeping the ciphertext identical across runs, this test shows how runtime decreases as more processes share the same total work.

The results reflect:

- Speedup: $S(np) = \frac{T(1)}{T(np)}$, where T is the runtime.
- Parallel Efficiency: $E(np) = \frac{S(np)}{np}$, where S is the speedup.

This allows for analysis of initialization and communication overhead, load-balancing, and behavior in the MPI implementation. The goal is to confirm that runtime decreases proportionally (approx.) with more ranks. This would show workload distribution across processes and identify where scaling begins to plateau due to communication overhead, synchronization costs, or load imbalance caused by the task allocation code which can be optimized when depth is adjusted for

This setup lets us analyze initialization cost, communication overhead, and load balance across ranks. The goal is to see runtimes drop (approx.) as ranks increase. Results should also reveal where scaling plateaus (from communication/synchronization overhead or load imbalance). In this implementation, the static, prefix-based task split can create idle workers; this can be improved by adjusting the decomposition depth ($-d$) or implementing a more dynamic work distribution such as finer-grained chunks or a sort of work-stealing method (see T11, fine-tuning $-d$).

Test Results and Observations:

```
{
  "name": "T10",
  "category": "Performance Scaling Analysis",
  "section": "Strong Scaling",
  "case": "Fixed ciphertext; np in [1,2,3,4,5,6,7,8]",
  "passed": true,
  "plaintext": "But we can rebuild.",
  "valgrind_enabled": false,
  "reps": 30,
  "detected_cores": 8,
  "total_ranks": 8,
  "ciphertext": "awt rb lni cbawued.",
  "n_letters": 12,
  "depth": 1,
  "expected_perms": 479001600,
  "visited_perms": 479001600,
  "valid_hits": 88,
  "found": [
    "[rank 0] [permutation: albwirundect] found: alb wi run dialect.",
    ...,
    "[rank 0] [permutation: butwecanrild] found: but we can rebuild.",
    ...
  ],
  "configs": [
    {
      "ranks": 1,
      "depth": 1,
      "times_sec": [27.232492, ..., 27.304457],
      "summary": {...}
    },
    ...,
    {
      "ranks": 8,
      "depth": 2,
      "times_sec": [4.726575, ..., 4.710904],
      "summary": {...}
    }
  ]
}
```

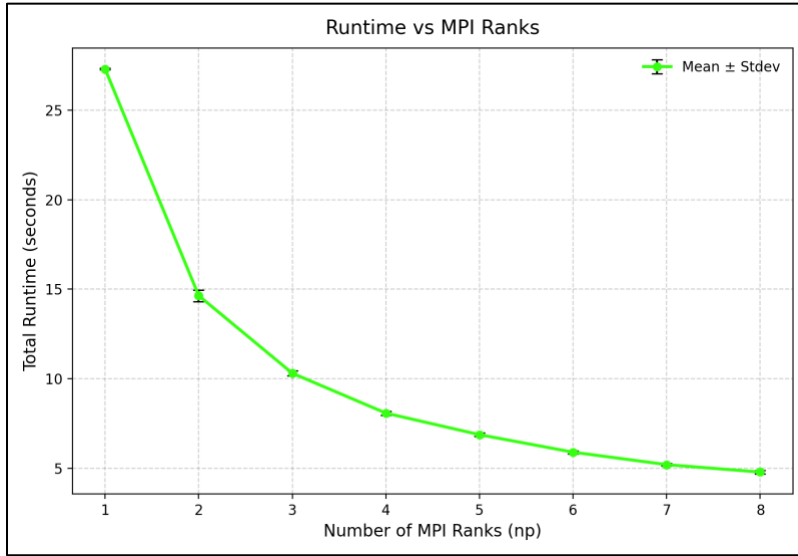


Figure 3: Runtime vs MPI Ranks (Mean and Standard Deviation)

As shown in Figure 3, total runtime decreases sharply as the number of MPI ranks increases from 1 to 8. The curve follows an expected strong-scaling trend, with the largest performance gain observed when moving from 1 to 2 ranks. Beyond that, the rate of improvement reduces slightly likely due to the communication and synchronization overhead. The low standard deviation across runs indicates stable performance and somewhat consistent workload distribution.

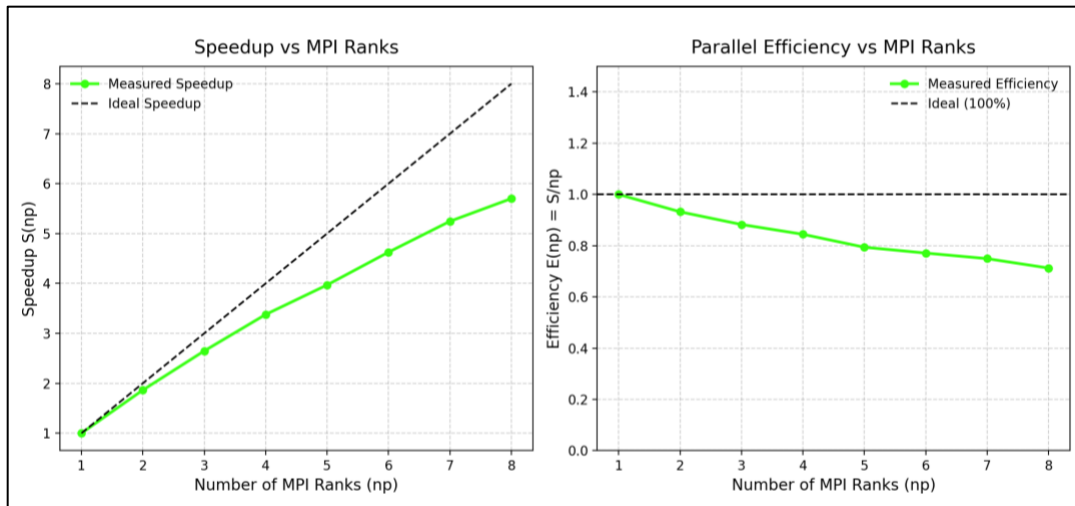


Figure 4: Speedup and Parallel Efficiency vs MPI Ranks

Figure 4 demonstrates the scaling behavior of the MPI program in terms of both speedup (4.1 left) and parallel efficiency (4.2 right). The measured speedup increases almost linearly up to $np = 8$, but it remains slightly below the ideal line. This confirms overall strong parallel performance with some communication and synchronization overhead. Parallel efficiency stays above 70% across all configurations, showing that adding more ranks continues to contribute effectively to the overall computation. The gradual decline in efficiency after 8 ranks is expected as the testing environment was on an 8-core machine, hence, inter-process communication costs would begin to dominate at higher ranks. In addition, there may be a slight improvement by fine tuning $-d$ (depth flag) for workload distribution as it is set to auto in these tests. This indicates the program scales but remains communication-bound at larger process/core counts.

Test 11 (Strong Scaling (Manual Depths d=1...10)):

Paths:

`tests/python/t11.py`

Description:

Similarly to T10 (strong scaling) but with a fixed prefix decomposition depth ($-d$ flag) per test and constant unique letters for the plaintext $n = 11$. Holding d constant may give us a better understanding of task-granularity vs. ranks, and how it affects runtime of the MPI decryption program: (1) If chunks are too coarse, some ranks idle may end up idle. (2) if chunks are too fine, overhead may dominate. While Test 10 automatically chose an appropriate depth (auto), this test locks the prefix depth to specific values ($d = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$). By controlling d , we can identify the optimal prefix depth that minimizes idle ranks without introducing excessive coordination cost. Smaller depths (ex: $d = 1$) create only a few large chunks, leading to load imbalance as ranks finish early. Larger depths (ex: $d = [9, 10]$) may yield many finer tasks, improving balance but increasing communication and prefix-enumeration overhead. For each decomposition, we run 15 iterations to measure variability and exclude the base case $np=1$ as task-granularity is irrelevant for serial decrypt. Thus, at each depth we run $np = [2, 3, 4, 5, 6, 7, 8]$. The goal is to speed up $S(np)$ and efficiency $E(np)$ like T10 and understand how d changes scaling.

Test Results and Observations: (Note: to view result files run `tests/python/t11.py`)

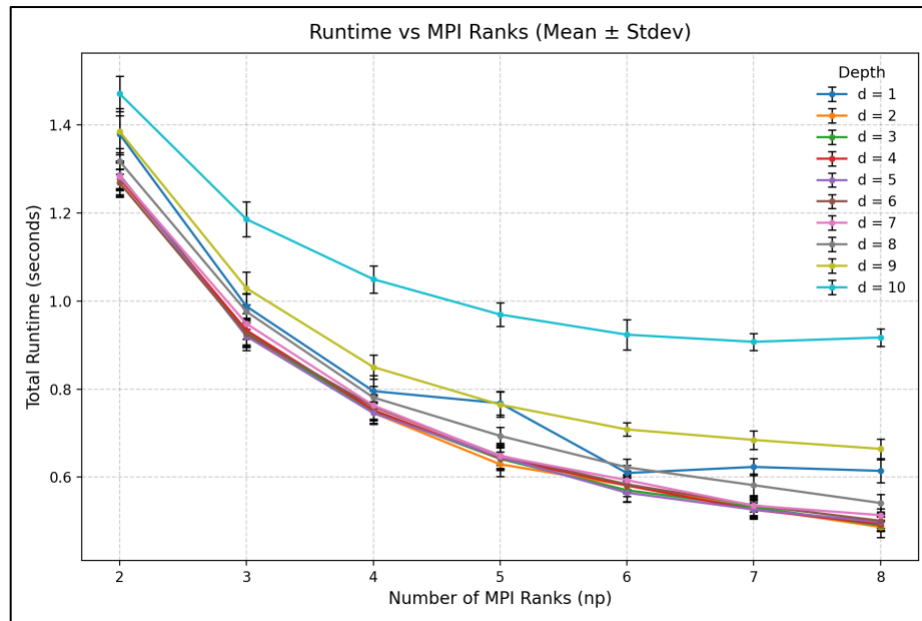


Figure 5: Runtime vs MPI Ranks

Figure 5 shows that runtime consistently decreases as the number of MPI ranks increases across all tested decomposition depths. Shallower depths ($d = 1$ to 2) surprisingly perform decently across ranks, but worse than the mid-range depths due to chunks being too coarse, likely leading to idle ranks once large chunks complete early. It is also worth noting the variance in $d = 1$ and the diminishing returns. In contrast, moderate depths ($d = 3$ to 7) achieve the best performance. They maintain balanced workloads with minimal synchronization overhead. Lastly, very deep decompositions ($d = 8$ to 10) perform the worst and begin to show diminishing returns ($d = 10$) as coordination costs dominate. The results show that an intermediate prefix depth provides optimal trade-off between parallel granularity and overhead.

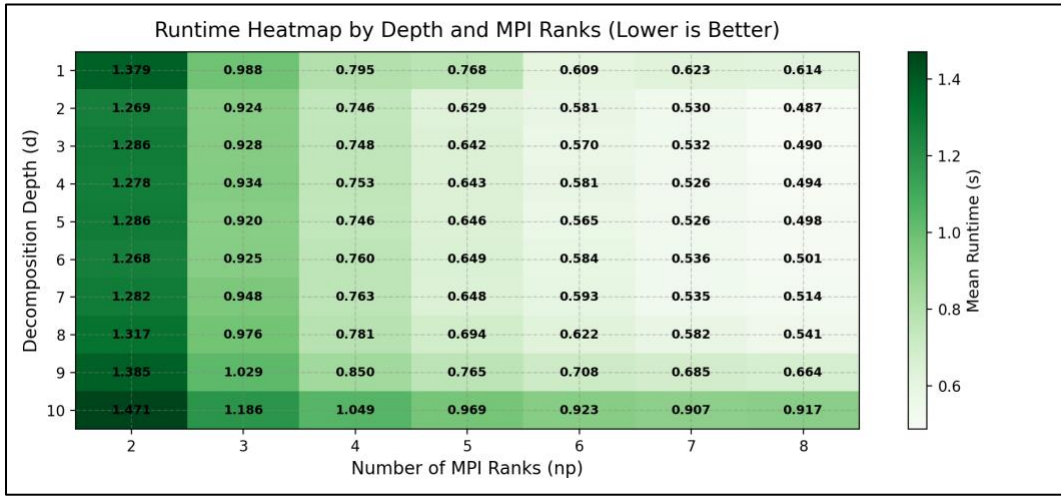


Figure 6: Runtime Heatmap

The heatmap in Figure 6 visually reinforces the previous results by showing how runtime changes with both decomposition depth and the number of MPI ranks. Darker regions indicate higher runtimes, which occur primarily at low ranks and extreme depths. The best-performing region is concentrated around depths $d = 3$ to 7 and ranks $np \geq 6$, where runtimes approach $\sim 0.5s$. This suggests that these configurations achieve near-optimal workload distribution with minimal idle time. As rank count increases, performance continues to improve but stabilizes near $np = 7$ to 8 . This might indicate that further parallelization yields (oversubscription past $np = 8$) limit gains due to MPI communication overhead and OS scheduling.

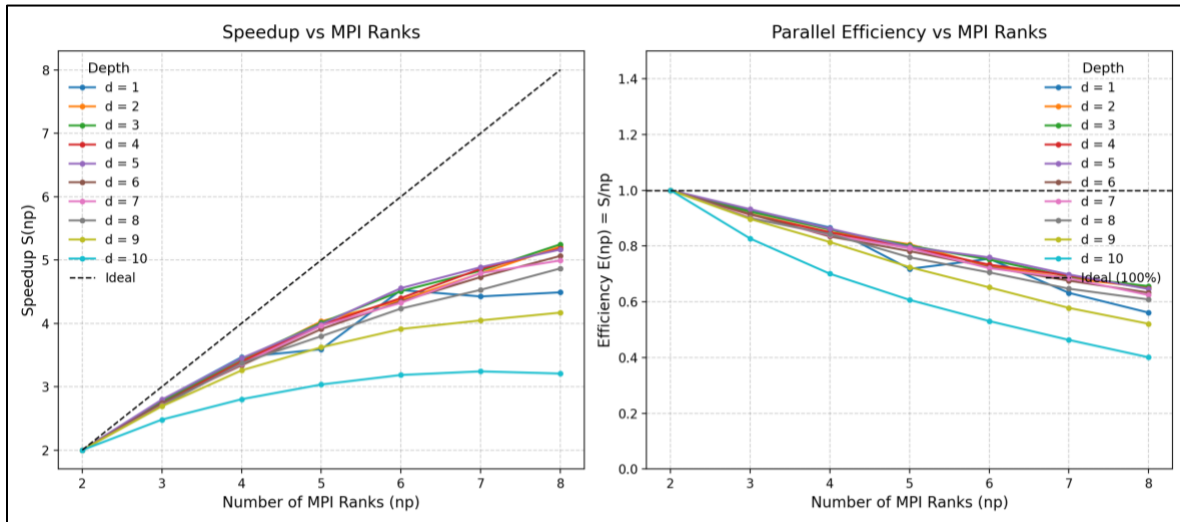


Figure 7: Speedup and Efficiency vs MPI Ranks

Figure 7 compares speedup and parallel efficiency across depths for different ranks. The speedup (7.1) curves show strong scaling up to eight ranks, with depths $d = 3$ to 7 achieves the highest speedsups, maintaining a close but increasing gap with the ideal linear speedup. Shallower and excessively deep decompositions both underperform. Parallel efficiency (7.2) remains above 60% for optimal depths and only drops significantly at extreme depths ($d = 9$ to 10). This also confirms that mid-level decomposition yields the most balanced and efficient scaling behavior.

Test 12 (To Use or Not to Use MPI):

Paths:

`tests/python/t12.py`

Description:

Test 12 evaluates when MPI parallelization becomes worthwhile by comparing serial and parallel (MPI) performance as the problem size (num unique letters n) grows.

For each $n = [2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]$, two benchmarks we're executed:

- The serial version ($np = 1$)
- The MPI version with a "perfect subscription" mapping ($np = [2, 3, 4, 5, 6, 7, 8, 8, 8, 8, 8]$).

Each configuration is run 30 times for variability. The goal of this test is to see if there is a break-even point where parallel startup and communication overhead are outweighed by the growing factorial search space $n!$ as n increases.

Test Results and Observations: (Note: to view result files run `tests/python/t12.py`)

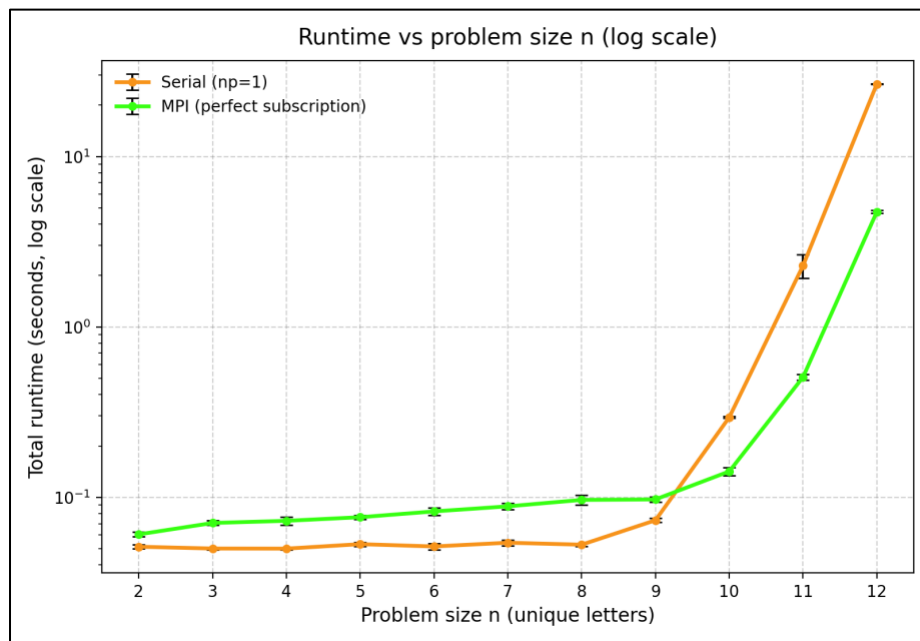


Figure 8: Runtime vs Problem Size (Log Scale)

Figure 8 shows how runtime scales exponentially with problem size n , consistent with the factorial growth of the permutation space $O(n!)$. A log scale was used to try and demonstrate the scaling. For small inputs ($n \leq 8$), both the serial and MPI versions perform similarly, with MPI slightly slower due to initialization and communication overhead. However, as n increases past 9, the MPI implementation begins to outperform the serial version. This marks the break-even point (using perfect-subscription and auto-depth) where the benefits of parallel workload distribution outweigh MPI's running costs. The increasing distance serial runtime from $n = 10$ to $n = 12$ shows that parallelization becomes more and advantageous for large combinatorial search spaces.

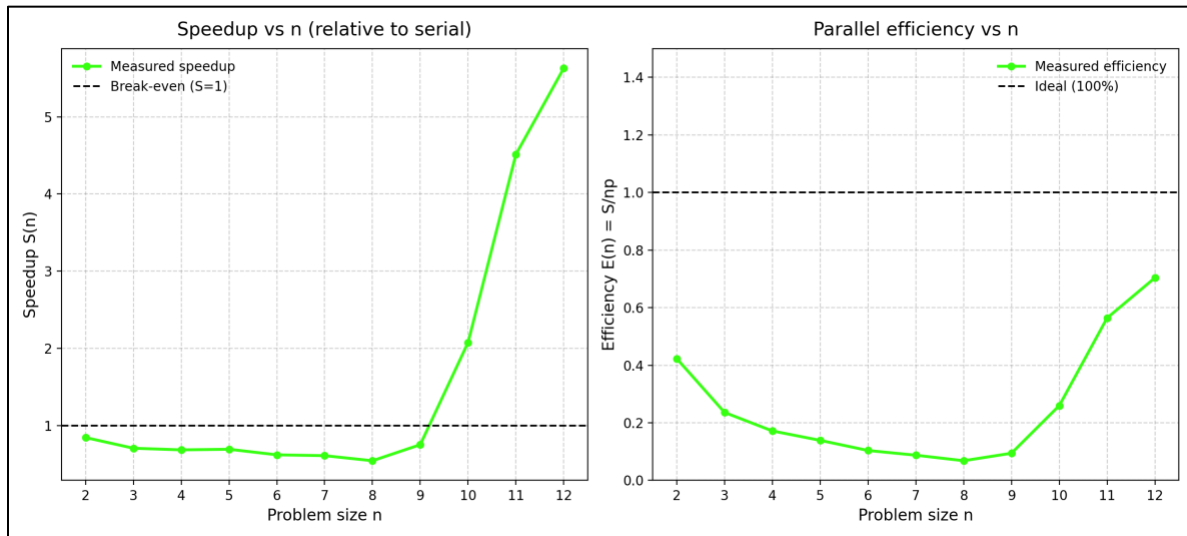


Figure 9: Speedup and Parallel Efficiency vs Problem Size

Figure 9 shows how speedup and efficiency evolve as the problem size (n) increases. The speedup curve (9.1 left) remains below 1 for smaller inputs, showing that MPI is initially slower due to various costs. However, At, $n = 10$, speedup surpasses the break-even, which may indicate the point where parallel execution becomes beneficial. The efficiency curve (9.2 right) follows a similar trend, there is a drop in efficiency from $n = 2$ to $n = 8$, but it then rises sharply after $n = 10$. In addition, it reflects better utilization of processes as the computational load increases. Which further suggests that the MPI implementation scales effectively for large problems, with diminishing efficiency losses as the workload size offsets communication, synchronization, and other overhead costs.

Conclusion

The performance analysis confirmed that the MPI implementation of the decryption program scales effectively as the number of ranks increases. The baseline results (T9) showed minimal difference (Serial performs slightly better) between the serial and MPI versions when run on a single process/core. This indicates low initialization overhead. The strong-scaling tests (T10) highlighted that runtime decreases sharply as we add more ranks, maintaining over 70% in parallel efficiency up to eight cores. Further depth variation tests (T11) showed that moderate decomposition depths ($d = 3$ to 7) provide the best balance between workload distribution and communication cost, while extreme depths on both ends cause diminishing returns and higher runtime. This test case is specific to my MPI implementation as there are different approaches to rank load-balancing. Finally, the break-even analysis (T12) showed that MPI becomes beneficial beyond $n = 9$ unique letters, where the factorial workload outweighs initialization and communication overheads. In conclusion, the results demonstrate that parallelization significantly improves performance for embarrassingly parallel problems and that it is best “To Use MPI” (T12) past. Reflecting on this project, I would have liked to have more time to experiment with oversubscription. My hypothesis is that it would not significantly improve performance beyond the available physical cores, since communication and context-switching overhead would likely offset any potential gains. However, it would be interesting to confirm this by running configurations with values where $np \geq \text{core_count}$ to see how the scheduler and MPI handled resources. In addition, I’d also like to explore pruning techniques or smarter workload distribution to reduce the search space (while searching) and further improve runtime.